C语言编程规范

（编制时间：2023.12.04）

编　　制：　　贺 俊 钢

审　　核：

批　　准：

成都航发机器人有限公司

CHENGDU HANGFA ROBOTICS CO,.LTD

各版本建立及修订履历

版本号		建立/修订履历	申请人/日期	批准人/日期
V1.00		初次建立，提交评审	贺俊钢	

目录

1 规范制订说明 6
1.1 前言 6
1.2 代码总体原则 6
1.2.1 清晰第一 6
1.2.2 简洁为美 6
1.2.3 选择合适的风格，与代码原有风格保持一致 6
2 文件整体结构 6
2.1 源文件 6
2.1.1 文件头注释 7
2.1.2 #include 部分 7
2.1.3 #define 部分 8
2.1.4 函数实现及注释 8
2.1.5 文件尾注释 8
2.2 头文件 9
2.2.1 头文件原则 9
2.2.2 文件头注释 10
2.2.3 文件尾注释 11
2.3 排版与格式 11
2.3.1 代码缩进 11
2.3.2 代码行相关规范 13
3 标识符命名规则 15
3.1 通用命名规则 15
3.2 文件命名规则 17
3.3 变量命名规则 17

3.4 函数命名规则 18
3.5 宏的命名规则 18
4 函数 18
4.1 一个函数仅完成一件功能 18
4.2 重复代码应该尽可能提炼成函数 18
4.3 避免函数过长，建议新增函数不超过 50 行（非空非注释行） 20
4.4 避免函数嵌套过深，新增函数的嵌套不超过 4 层 20
4.5 可重入函数应避免使用共享变量 22
4.6 对参数的合法性检查 23
4.7 对函数的错误返回码要全面处理 23
4.8 废弃代码（没有被调用的函数和变量）要及时清除 24
4.9 函数的参数个数不超过 5 个 24
4.10 函数不变参数使用 const 24
5 变量 25
5.1 一个变量只能有一个功能 25
5.2 不用或者少用全局变量 25
5.3 防止局部变量与全局变量同名 26
5.4 严禁使用未经初始化的变量作为右值 26
5.5 尽量减少没有必要的数据类型默认转换与强制转换 26
6 宏、常量 27
6.1 用宏定义表达式时，要使用完备的括号 27
6.2 禁止使用宏定义语句 27
6.3 使用宏时，不允许参数发生变化 27
6.4 常量建议使用 const 定义代替宏 27
7 注释 28
7.1 注释的内容要清楚、明了，含义准确，防止注释二义性 28

7.2 在代码的功能、意图层次上进行注释 28
7.3 修改代码时，维护代码周边的所有注释，保证一致性 28
7.4 全局变量要详细的注释 28
7.5 注释应放在其代码上方相邻位置或右方，规定使用/**/注释。 28
7.6 switch 语句注释 29
7.7 避免在注释中使用缩写，除非是业界通用或子系统内标准化的缩写 29

1 规范制订说明

1.1 前言

为提高代码质量，规范开发人员在各种格式上一致，便于检查与复用，以提高编程效率、编写出简洁、可维护、可靠、可测试、高效、可移植的代码。

1.2 代码总体原则

1.2.1 清晰第一

清晰性是易于维护、易于重构的程序必需具备的特征。

1.2.2 简洁为美

简洁就是易于理解并且易于实现。

1.2.3 选择合适的风格，与代码原有风格保持一致

如果重构/修改其他风格的代码时，比较明智的做法是根据现有代码的现有风格继续编写代码

2 文件整体结构

2.1 源文件

对于源码文件中的段落结构，建议按如下的顺序排列：

- 文件头注释
- #include 部分
- #define 部分
- enum 常量定义
- 类型定义，包括 struct、union、typedef 等
- 全局变量定义
- 文件级变量定义
- 全局或文件级函数声明
- 函数实现及注释
- 文件尾注释

2.1.1 文件头注释

文件头部应进行注释，注释必须列出：版权说明、版本号、生成日期、作者姓名、功能说明、与其它文件的关系等。

```
/********************************** Copyright ***********************************
  *
  *                          成都航发机器人有限公司
  *                           All Rights Reserved
  *
  * FileName    : hello.c
  * Version     : V1.0
  * Author      : hejungang
  * Date        : 2023.12.04
  * Description: {描述}
  ********************************************************************************/
```

2.1.2 #include 部分

(1) 在引用头文件时，不要使用绝对路径，使用相对路径，否则当需要移动目录时，必须修改所有相关代码，繁琐且不安全。

例如：

```
~~#include "/project/inc/hello.h"~~     /* 不应使用绝对路径 */
#include "../inc/hello.h"              /* 可以使用相对路径 */
```

(2) 在引用头文件时，使用#include" " 来包含用户创建的头文件，使用#include< > 来包含库文件

```
#include   <stdio.h>          /* 标准头文件 */
#include   <projdefs.h>       /* 工程指定目录头文件 */
#include   "global.h"         /* 当前目录头文件 */
#include   "inc/config.h"     /* 路径相对于当前目录的头文件 */
```

2.1.3 #define 部分

定义宏时，应注意对齐，使代码更美观、简洁，增加可读性。

```
#define   RECTANGLE_AREA(a, b)        ((a) * (b))
#define   MAX_NODE_NUM                10
#define   ASPECT_RATIO                1.653f
```

2.1.4 函数实现及注释

(1) 对于函数，在函数实现之前，应该给出和函数的实现相关的足够而精练的注释信息。内容包括本函数功能介绍，调用的变量、常量说明，形参说明，特别是全局、全程或静态变量，要求对其初值，调用后的预期值作详细的阐述。

(2) 边写代码边注释，修改代码同时修改相应的注释，以保证注释与代码的一致性。不再有用的注释要删除。

(3) 注释的内容要清楚、明了，含义准确，防止注释二义性。

```
/*************************************************************************
* 函数名:  SendToCard()
* 功   能: 向读卡器发命令，如果读卡器进入休眠，则首先唤醒它
* 参   数: 全局变量 &gaTxCard     存放待发的数据
*          全局变量 gbTxCardLen  存放长度
* 返回值: 无
*************************************************************************/
```

2.1.5 文件尾注释

建议在文件尾部添加如下注释，并在注释下方留一空行。

```
/*************************************END OF FILE*************************************/
```

2.2 头文件

对于头文件中的段落结构，建议按如下的顺序排列：

- 文件头注释
- #include 部分
- #define 部分
- enum 常量声明
- 类型声明，包括 struct、union、typedef 等
- 全局变量声明
- 全局函数声明
- 文件尾注释

2.2.1 头文件原则

(1) 头文件中适合放置接口的声明，不适合放置实现

说明：头文件是模块（Module）或单元（Unit）的对外接口。头文件中应放置对外部的声明，如对外提供的函数声明、宏定义、类型定义等。

内部使用的函数（相当于类的私有方法）声明不应放在头文件中。

内部使用的宏、枚举、结构定义不应放入头文件中。

变量定义不应放在头文件中，应放在.c 文件中。

变量的声明尽量不要放在头文件中，亦即尽量不要使用全局变量作为接口。变量是模块或单元的内部实现细节，不应通过在头文件中声明的方式直接暴露给外部，应通过函数接口的方式进行对外暴露。即使必须使用全局变量，也只应当在.c 中定义全局变量，在.h 中仅声明变量为全局的。

(2) 头文件应当职责单一

说明：头文件过于复杂，依赖过于复杂是导致编译时间过长的主要原因。很多现有代码中头文件过大，职责过多，再加上循环依赖的问题，可能导致为了在.c 中使用一个宏，而包含十几个头文件。导致在编译过程中，某些重复头文件被反复展开，大大增加了工程的编译时间。

(3) 每一个.c 文件应有一个同名.h 文件，用于声明需要对外公开的接口

(4) .c/.h 文件禁止包含用不到的头文件

(5) 总是编写内部#include 保护符（#define 保护）

定义包含保护符时，应该遵守如下规则：

1) 保护符使用唯一名称；

2) 不要在受保护部分的前后放置代码或者注释。

示例：如 timer.h

```
#ifndef    _TIMER_H_
#define    _TIMER_H_
...
#endif
```

(6) 函数声明下方应包含使用方法、参数类型、返回值等详细信息。

```
/*************************************************************************
* 函数名:bool SendToCard(u8* TxCard , u16 TxCardLen)
* 功   能: 向读卡器发命令，如果读卡器进入休眠，则首先唤醒它
* 参   数: 全局变量 &TxCard     存放待发的数据
*          全局变量 TxCardLen  存放长度
* 返回值: 成功/失败
*************************************************************************/
```

2.2.2 文件头注释

文件头部应进行注释，注释必须列出：版权说明、版本号、生成日期、作者姓名、与其它文件的关系等。

```
/********************************** Copyright **********************************
 *
 *                          成都航发机器人有限公司
 *                           All Rights Reserved
 *
 * FileName    : hello.h
 * Version     : V1.0
 * Author      : hejungang
 * Date        : 2023.12.04
 * Description: {描述}
 ******************************************************************************/
```

2.2.3 文件尾注释

建议在文件尾部添加如下注释，并在注释下方留一空行。

```
/************************************END OF FILE************************************/
```

2.3 排版与格式

文本编码标准为 GB2312

2.3.1 代码缩进

代码缩进要使用制表符，也就是 TAB 键，不要使用空格键缩进！一个 TAB 为 4 个空格。（宏定义、编译开关、条件预处理语句可以顶格）

(1) if、for、do、while、case、switch、default 等语句独占一行

#“ swich”和 “case”标签应换行，且 case 标签需缩进一个 TAB。

#“case”与标签之间应间隔一个空格。

#“case”标签后的语句都需要换行缩进！

每个“case”间需增加一行空行。

每个“case”必须有“break;”，除非需要顺序执行。

switch 中应有“default”标签，以处理各种预期外的情况。

有 if 、else 必须成对出现。

如下所示：

```
if ((temp_flag_var == TEST_FLAG)
 && (((temp_counter_var - TEST_COUNT_BEGIN) % TEST_COUNT_MODULE) >= TEST_COUNT_THRESHOLD))
{
    // process code
}
else
{
    __nop;//无代码的情况
}

if ((temp_flag_var == TEST_FLAG)
 && (((temp_counter_var - TEST_COUNT_BEGIN) % TEST_COUNT_MODULE) >= TEST_COUNT_THRESHOLD))
{
    // process code
}
else
{
    // process code//有代码的情况
}
```

```
switch(g_mc_step)
{
    case MC_Init:
    {
        //program code
        ...
    }
    break;

    case MC_Check;
    {
        //program code
        ...
    }
    break;

    ...

    default:
        break;
}
```

2.3.2 代码行相关规范

(1) 一条语句不能过长，如不能拆分需要分行写。一行 130 字符换行比较合适。

换行时有如下建议：

- 换行时要增加一级缩进，使代码可读性更好；
- 低优先级操作符处划分新行；换行时操作符应该也放下来，放在新行首；
- 换行时建议一个完整的语句放在一行，不要根据字符数断行。

示例：

(2) 相对独立的程序块之间、变量说明之后必须加空行。

示例：

```
if (!valid_ni(ni))
{
    // program code
    ...
}
repssn_ind = ssn_data[index].repssn_index;            //不符合规范
repssn_ni = ssn_data[index].ni;


if (!valid_ni(ni))
{
    // program code
    ...
}

repssn_ind = ssn_data[index].repssn_index;            //符合规范
repssn_ni = ssn_data[index].ni;
```

(3) 多个短语句（包括赋值语句）不允许写在同一行内，即一行只写一条语句。

示例:

```
不好的排版：
    int a = 5; int b = 10;
    float c = 1.0; float d = 2.0;

较好的排版：
    int a = 5,
        b = 10;

    float c = 1.0,
          d = 2.0;
```

(4) 在两个以上的关键字、变量、常量进行对等操作时，它们之间的操作符之前、之后或者前后要加空格；进行非对等操作时，如果是关系密切的立即操作符（如->），后不应加空格。

采用这种松散方式编写代码的目的是使代码更加清晰.

逗号、分号只在后面加空格

```
int a, b, c;
```

比较操作符，赋值操作符"="、 "+="，算术操作符"+"、"%"，逻辑操作符"&&"、"&"，位域操作符"<<"、"^"等双目操作符的前后加空格

```
if (current_time >= MAX_TIME_VALUE)
a = b + c;
a *= 2;
a = b ^ 2;
```

"!"、"~"、"++"、"--"、"&"（地址操作符）等单目操作符前后不加空格

```
*p = 'a';
Flag = !is_empty;
p = &mem;
i++;
```

"->"、"."前后不加空格

```
p->id = pid;
Mc_Data.id = pid;
```

3 标识符命名规则

3.1 通用命名规则

统一的命名风格可以提高源码的易读性和可维护性，帮助团队开发出风格统一的代码，使代码更加整洁。

驼峰命名法：

驼峰命名法应该我们最常见的一个，这种命名方式使用大小写混合的格式来区别各个单词，并且单词之间不使用空格隔开或者连接字符连接的命名方式。其中，驼峰命名法包含“大驼峰命名法”和“小驼峰命名法”,规定使用“大驼峰命名法”。

1. 大驼峰式(Upper Camel Case)：每一个单词的首字母都采用大写字母，如MyClass。

大驼峰式：**void TskMotionControl （void）** **McData** **SetMotorEnable**

(1) 标识符的命名要清晰、明了，有明确含义，同时使用完整的单词或大家基本可以理解的缩写，避免使人产生误解，尽可能给出描述性名称，不要节约空间，让别人很快理解你的代码更重要。

(2) 除了常见的通用缩写以外，不使用单词缩写，不得使用汉语拼音。

示例：一些常见可以缩写的例子：

argument 可缩写为 Arg

buffer 可缩写为 Buff

clock 可缩写为 Clk

command 可缩写为 Cmd

compare 可缩写为 Cmp

configuration 可缩写为 Cfg

device 可缩写为 Dev

error 可缩写为 Err

hexadecimal 可缩写为 Hex

increment 可缩写为 Inc

initialize 可缩写为 Init

maximum 可缩写为 Max

message 可缩写为 Msg

minimum 可缩写为 Min

parameter 可缩写为 Para

previous 可缩写为 Prev

register 可缩写为 Reg

semaphore 可缩写为 Sem

statistic 可缩写为 Stat

synchronize 可缩写为 Sync

temp 可缩写为 Tem

(3) 应保持统一的命名风格。

(4) 建议用正确的反义词组命名具有互斥意义的变量或相反动作的函数等。

示例：

add/remove	begin/end	create/destroy
insert/delete	first/last	get/release
increment/decrement	put/get	add/delete
lock/unlock	open/close	min/max
old/new	start/stop	next/previous
source/target	show/hide	send/receive
source/destination	copy/paste	up/down

(5) 尽量避免名字中出现数字编号，除非逻辑上的确需要编号。

(6) 重构/修改部分代码时，应保持和原有代码的命名风格一致。

3.2 文件命名规则

除文件名前缀采用特点名称，后续应符合“大驼峰命名法”。

驱动层前缀：drv_Xxxx.c

应用层前缀：app_Xxxx.c

3.3 变量命名规则

1. 全局变量应增加“g_”前缀。
2. 静态变量应增加“s_”前缀。
3. 全局静态变量应增加“g_”前缀。
4. 指针变量应增加“p_”前缀。
5. const 常量不需要前缀。
6. 禁止使用单字节命名变量，但允许定义 i、j、k 作为局部循环变量。

7. 使用名词或者形容词＋名词方式命名变量。

3.4 函数命名规则

1. 函数命名应以函数要执行的动作命名，一般采用动词或者动词＋名词的结构。

   示例：找到当前进程的当前目录

   DWORD GetCurrentDirectory(DWORD BufferLength, LPTSTR Buffer);

2. 函数指针除了前缀，其他按照函数的命名规则命名。

   示例：u8 (*SetMotorBreak)(u8 MotorID, u8 Value);

3.5 宏的命名规则

1. 对于数值或者字符串等等常量的定义，建议采用全大写字母，单词之间加下划线„_"的方式命名（枚举同样建议使用此方式定义）。

```
#define PI_ROUNDED (3.14f)
```

4 函数

4.1 一个函数仅完成一件功能

一个函数实现多个功能给开发、使用、维护都带来很大的困难。将没有关联或者关联很弱的语句放到同一函数中，会导致函数职责不明确，难以理解，难以测试和改动。

4.2 重复代码应该尽可能提炼成函数

重复代码提炼成函数可以带来维护成本的降低，一般情况下，可以通过提炼函数的形式消除重复代码。

示例：

```
char* Function(char * buf)
{
    …… 
```

```
if(xxx)
{
    if(xxx)
    {
        struct.member_1 = 0;
        struct.member_2 = 0;
        struct.member_3 = 0;
    }
}
else if(xxx)
{
    if(xxx)
    {
        struct.member_1 = 0;
        struct.member_2 = 0;
        struct.member_3 = 0;
    }
}
if(xxx)
{
    if(xxx)
    {
        if(xxx)
        {
            struct.member_1 = 0;
            struct.member_2 = 0;
            struct.member_3 = 0;
```

```
            }
        }
    }
}
```

红色字体的部分是简单的代码重复，粗体字部分是代码结构的重复，将重复部分提炼成一个函数即可消除重复

4.3 避免函数过长，建议新增函数不超过 50 行（非空非注释行）

过长的函数往往意味着函数功能不单一，过于复杂。函数的有效代码行数，即NBNC（非空非注释行）应当在[1，50]区间。

4.4 避免函数嵌套过深，新增函数的嵌套不超过 4 层

函数的代码块嵌套深度指的是函数中的代码控制块（例如：if、for、while、switch等）之间互相包含的深度。每级嵌套都会增加阅读代码时的脑力消耗，因为需要在脑子里维护一个“栈”（比如，进入条件语句、进入循环,,,,）。应该做进一步的功能分解，从而避免使代码的阅读者一次记住太多的上下文。优秀代码参考值：[1, 4]。

示例：

```
void serial (void)
{
    if (!Received)
    {
        TmoCount = 0;
        switch (Buff)
        {
            case AISGFLG:
              if ((TiBuff.Count > 3))
                {
                    if(Flg7E == false)
                    {
                        Received = true;
```

```
                    }
                    else
                    {
                        Received = true;
                        TiBuff.Count = 0;
                        if(TiBuf.Buff[0] == CurPa.ADDR)
                        {
                            Received = true;
                        }
                    }
                }
                else
                {
                    TiBuff.Count = 0;
                    Flg7D = false;
                    Flg7E = true;
                }
                break;
            default:
                break;
        }
    }
}
```

如上代码嵌套深度为 5，可将“AISGFLG”标签中的“if”“eles”语句单独写成一个函数，来减少嵌套深度。

4.5 可重入函数应避免使用共享变量

可重入函数是指可能被多个任务并发调用的函数。若需要使用，则应通过互斥手段（关中断、信号量）对其加以保护。在 RTOS 中（如 RTX5、FreeRTOS），函数具有可重入性是多个任务可以共用此函数的必要条件。共享变量指的全局变量和 static 变量。

编写 C 语言的可重入函数时，不应使用 static 局部变量，否则必须经过特殊处理，才能使函数具有可重入性。

示例：函数 square_exam 返回 g_exam 平方值。那么如下函数不具有可重入性。

```
int g_exam;
unsigned int example( int para )
{
    unsigned int temp;
    g_exam = para; //  （**）
    temp = square_exam ( );
    return temp;
}
```

此函数若被多个线程调用的话，其结果可能是未知的，因为当（**）语句刚执行完后，另外一个使用本函数的线程可能正好被激活，那么当新激活的线程执行到此函数时，将使 g_exam 赋于另一个不同的 para 值，所以当原线程重新回到“temp =square_exam ( )”后，计算出的 temp 很可能不是预想中的结果。此函数应如下改进。

```
int g_exam;
unsigned int example( int para )
{
    unsigned int temp;
    if(申请信号量)
    {                                   //若申请到信号量
        g_exam = para;
        temp = square_exam( );
```

```
        [释放信号量];
    }
    else
        return temp;
}
```

若申请不到“信号量”，说明另外的进程正处于给 g_exam 赋值并计算其平方过程中（即正在使用此信号），本进程必须等待其释放信号后，才可继续执行。其它线程必须等待本线程释放信号量后才能再使用本信号。

4.6 对参数的合法性检查

由接口函数负责。

对于模块间接口函数的参数的合法性检查这一问题，往往有两个极端现象，即：要么是调用者和被调用者对参数均不作合法性检查，结果就遗漏了合法性检查这一必要的处理过程，造成问题隐患；要么就是调用者和被调用者均对参数进行合法性检查，这种情况虽不会造成问题，但产生了冗余代码，降低了效率。

```
int ServiceProcess(int CbNo, MsgBlock *Msg)
{
    if ( Msg == NULL)
    {
        return 1;
    }
    … …
    // 业务处理代码
    … …
    return 0;
}
```

4.7 对函数的错误返回码要全面处理

一个函数一般都会提供一些指示错误发生的方法，一般都用返回值来表示，因此我们必须对这些错误标识做处理。

4.8 废弃代码（没有被调用的函数和变量）要及时清除

程序中的废弃代码不仅占用额外的空间，而且还常常影响程序的功能与性能，很可能给程序的测试、维护等造成不必要的麻烦。

4.9 函数的参数个数不超过 5 个

函数的参数过多，会使得该函数易于受外部（其他部分的代码）变化的影响，从而影响维护工作。函数的参数过多同时也会增大测试的工作量。函数的参数个数不要超过 5 个，如果超过了建议拆分为不同函数。

4.10 函数不变参数使用 CONST

不变的值更易于理解/跟踪和分析，把 const 作为默认选项，在编译时会对其进行检查，使代码更牢固/更安全

示例：C99 标准 7.21.4.4 中 strncmp 的例子，不变参数声明为 const。

```
int strncmp(const char *s1, const char *s2, register size_t n)
{
    register unsigned char u1, u2;
    while (n-- >    0)
    {
        u1 = (unsigned char) *s1++;
        u2 = (unsigned char) *s2++;
        if (u1 != u2)
        {
            return u1 - u2;
        }
        if (u1 == '\0')
        {
            return 0;
        }
```

```
        }
        return 0;
    }
```

5 变量

5.1 一个变量只能有一个功能

一个变量只能有一个特定功能，不能把一个变量作为多用途使用，即一个变量取值不同的时候其代表的含义也不同，如：

```
WORD DelRelTimeQue(void)
{
    WORD Locate;
    Locate = 3;
    Locate = DeleteFromQue(Locate);
    return Locate;
}
```

上述函数中，Locate 变量即做位置变量，又做返回值，违背了一个变量只能有一个功能的原则，应改为：

```
WORD DelRelTimeQue(void)
{
    WORD Ret;
    WORD Locate;
    Locate = 3;
    Ret = DeleteFromQue(Locate);
    return Ret;
}
```

5.2 不用或者少用全局变量

单个文件内部可以使用 static 的全局变量，可以将其理解为类的私有成员变量。全局变量应该是模块的私有数据，不能作用对外的接口使用，使用 static 类型定义，可以有效防止外部文件的非正常访问，建议定义一个 STATIC 宏，在调试阶段，将 STATIC 定义为 static，版本发布时，改为空。

```
#ifdef _DEBUG
#define STATIC static
#else
#define STATIC
#endif
```

直接访问其它模块的私有数据，会增强模块之间的耦合关系。

5.3 防止局部变量与全局变量同名

尽管局部变量和全局变量的作用域不同而不会发生语法错误，但容易使人误解。

5.4 严禁使用未经初始化的变量作为右值

```
/* 不可取的初始化：无意义 * /
int num = 2;
if(a) num = 3;
num=4;
 /* 不可取的初始化：初始化和声明分离 */
int num;
if(a) num = 3;
else num=4;
/* 较好的初始化：使用默认有意义的初始化 */
int num = 3;
if(a) num = 4;
/* 较好的初始化：  ：?:减少数据流和控制流的混合 */
int num=(a?4:3);
```

5.5 尽量减少没有必要的数据类型默认转换与强制转换

当进行数据类型强制转换时，其数据的意义、转换后的取值等都有可能发生变化，而这些细节若考虑不周，就很有可能留下隐患.

6 宏、常量

6.1 用宏定义表达式时，要使用完备的括号

因为宏只是简单的代码替换，不会像函数一样先将参数计算后，再传递，若不完备括号，在宏展开时，由于运算符的优先级不同，可能会造成计算顺序与预期不符。

示例：如下定义的宏都存在一定的风险

```
#define RECTANGLE_AREA(a, b)     a * b
#define RECTANGLE_AREA(a, b)     (a * b)
#define RECTANGLE_AREA(a, b)     (a) * (b)
```

正确的定义应为：

```
#define RECTANGLE_AREA(a, b)     ((a) * (b))
```

6.2 禁止使用宏定义语句

6.3 使用宏时，不允许参数发生变化

示例：如下用法可能导致错误。

```
#define SQUARE(a) ((a) * (a))
int a = 5;
int b;
b = SQUARE(a++); //  结果：a = 7，即执行了两次增。
```

正确的用法是：

```
b = SQUARE(a);
a++;                    //  结果：a = 6，即只执行了一次增。
```

6.4 常量建议使用 CONST 定义代替宏

```
const double ASPECT_RATIO = 1.653;
```

这种方法很有效，但有两个特殊情况要注意。首先，定义指针常量时会有点不同。因为常量定义一般是放在头文件中（许多源文件会包含它），除了指针所指的类

型要定义成 const 外，重要的是指针也经常要定义成 const。例如，要在头文件中定义一个基于 char*的字符串常量，你要写两次 const：

```
const char * const authorName = "Scott Meyers";
```

7 注释

7.1 注释的内容要清楚、明了，含义准确，防止注释二义性

7.2 在代码的功能、意图层次上进行注释

注释的目的是解释代码的目的、功能和采用的方法，提供代码以外的信息，帮助读者理解代码，防止没必要的重复注释信息。

对于实现代码中巧妙的、晦涩的、有趣的、重要的地方加以注释。

注释不是为了名词解释（what），而是说明用途（why）。

7.3 修改代码时，维护代码周边的所有注释，保证一致性

不要将无用的代码留在注释中，随时可以从源代码配置库中找回代码；即使只是想暂时排除代码，也要留个标注，不然可能会忘记处理它。

7.4 全局变量要详细的注释

全局变量要有较详细的注释，包括对其功能、取值范围以及存取时注意事项等的说明。

7.5 注释应放在其代码上方相邻位置或右方，规定使用/**/注释。

注释应放在其代码上方相邻位置或右方，不可放在下面。如放于上方则需与其上面的代码用空行隔开，且与下方代码缩进相同。

示例：

```
/* active statistic task number */

#define MAX_ACT_TASK_NUMBER 1000
```

```
#define MAX_ACT_TASK_NUMBER 1000 /* active statistic task number */
```

可按如下形式说明枚举/数据/联合结构。

```
/* sccp interface with sccp user primitive message name */
enum SCCP_USER_PRIMITIVE
{
    N_UNITDATA_IND, /* sccp notify sccp user unit data come */
    N_NOTICE_IND, /* sccp notify sccp user unit data come */
    N_UNITDATA_REQ, /* sccp user's unit data transmission request*/
};
```

7.6 SWITCH 语句注释

对于 switch 语句下的 case 语句，如果因为特殊情况需要处理完一个 case 后进入下一个 case 处理，必须在该 case 语句处理完、下一个 case 语句前加上明确的注释

这样比较清楚程序编写者的意图，有效防止无故遗漏 break 语句。

示例（注意斜体加粗部分）：

```
case CMD_FWD:
    ProcessFwd();
    /* now jump into case CMD_A */
case CMD_A:
    ProcessA();
    break;
```

7.7 避免在注释中使用缩写，除非是业界通用或子系统内标准化的缩写